Title: A 32-bit RISC-V ASIC implementation using Chisel

A DISSERTATION SUBMITTED IN PARTIAL FULFILMENT
FOR THE DEGREE OF

Master of Technology

IN THE
FACULTY OF ENGINEERING

BY

SANIDHYA SAXENA, TOMS JIJI VARGHESE

GUIDED BY

KURUVILLA VARGHESE

DEPARTMENT OF ELECTRONIC SYSTEMS ENGINEERING

INDIAN INSTITUTE OF SCIENCE, BANGALORE

MAY 2024

Contents

Chapter 1

Pre-study

This report details the pre-study conducted to understand the design and architecture of a RISC-V processor and its implementation using the Chisel hardware construction language. We embarked on a comprehensive exploration through:

- **Market Survey:** We investigated existing RISC-V-based controllers to identify industry trends, potential application areas, and current performance benchmarks.
- **Product Survey:** We analyzed commercially available controllers (if applicable) to understand their functionalities, resource utilization, and potential limitations.
- **Wish List Specification:** Based on the market and product surveys, we formulated a detailed wish list outlining the desired features and capabilities of our RISC-V controller.

1.1 Chisel: A Powerful Tool for Hardware Design

The pre-study identified Chisel as a compelling open-source hardware description language (HDL) for constructing digital circuits and electronics at the register-transfer level (RTL). Chisel stands for "Constructing Hardware in a Scala Embedded Language," reflecting its innovative approach of leveraging Scala syntax for hardware design. This offers several advantages:

- **Expressive Syntax**: Chisel inherits the expressiveness and flexibility of Scala, allowing for concise and readable hardware descriptions. Compared to traditional HDLs like Verilog or VHDL, Chisel can simplify complex hardware constructs, improving design clarity and maintainability.

- **Functional Programming Paradigm**: Chisel embraces the functional programming paradigm, promoting immutability and promoting hardware components as functions of their inputs. This fosters a more modular design approach, making it easier to decompose and reason about complex digital systems.
- **Integration with Scala Ecosystem**: Chisel seamlessly integrates with the rich Scala ecosystem, allowing developers to leverage existing libraries and tools for tasks like testing, verification, and simulation. This can streamline the hardware design process and enhance overall development efficiency.

To effectively utilize Chisel's capabilities, a development environment was established on a Linux operating system. This involved installing and configuring essential tools like:

- **Scala Compiler**: The Scala compiler is required to translate Chisel code into its underlying hardware representation.
- **Chisel Libraries**: Specific Chisel libraries provide essential building blocks and functionalities for hardware design.
- **Simulation Tools**: Simulation tools like FIRRTL allow for testing and verification of Chisel designs before hardware synthesis.

Following the establishment of the development environment, the pre-study progressed to focus on the design of fundamental digital building blocks. These elementary components constitute the cornerstone upon which more intricate digital systems are constructed.

- **Counters**: These circuits execute incrementing or decrementing operations, serving a critical role in functionalities such as timing and sequencing within a digital system.
- **Multiplexers**: Multiplexers possess the capability of selecting a single data input from a collection of potential inputs based on a designated control signal. This functionality allows for flexible data routing within the system.
- **Finite State Machines (FSMs)**: FSMs are sequential circuits that transition between distinct states in response to both input and output signals. They play a central role in numerous digital systems, forming the core logic for controlling their behavior.

The process of designing and implementing these fundamental elements not only facilitated a comprehensive comprehension of Chisel's functionalities but also established a firm grasp of the core principles that govern digital hardware design.

1.2 Market Survey

RISC-V (RISC Five), an open-source instruction set architecture (ISA) is rapidly transforming the computing landscape. Unlike traditional proprietary ISAs, RISC-V offers a royalty-free and modular standard. This fosters a wave of innovation across diverse computing domains, including the Internet of Things (IoT), edge computing, and Artificial Intelligence (AI). RISC-V's low-power consumption and inherent scalability make it ideal for resource-constrained IoT devices. Additionally, its flexible nature allows for customization to specific processing needs encountered at the edge of computing networks, and its modularity facilitates the development of specialized hardware accelerators for AI applications.

1.2.1 Market Trends

The pre-study revealed a burgeoning market for RISC-V processors, driven by several key trends:

- **Soaring Adoption**: Across diverse sectors, the open nature of RISC-V is fueling rapid adoption. This openness allows for customization, reduces time to market, and provides cost savings for developers.
- **Application Expansion**: Initially popular in embedded systems, RISC-V is now making inroads into high-performance computing domains such as servers, networking equipment, and even some mobile devices. This diversification showcases its growing versatility.
- **Thriving Ecosystem**: The RISC-V ecosystem is flourishing with the development of essential tools like software tools, compilers, operating systems, and hardware platforms. This robust ecosystem fosters innovation and further adoption.
- **Industry Backing**: Major tech giants, including Google, NVIDIA, Western Digital, and Alibaba, are lending their support to RISC-V. This industry backing signifies its potential to become a mainstream architecture in the future.

1.2.2 Growing Demand for Customization:

RISC-V's open architecture is a key driver behind its growing adoption. This openness unlocks a new level of customization for companies developing Application-Specific Integrated Circuits

(ASICs). Beyond the traditional benefits of ASICs, RISC-V empowers a more granular level of control, allowing for:

- **Tailored Solutions**: Traditional ASICs offer customization, but RISC-V's open architecture unlocks a new level. Companies can fine-tune their ASIC designs, optimizing performance, power efficiency, and other critical metrics to their exact application requirements.
- **Industry-Specific Optimization**: Different industries have unique needs. RISC-V empowers companies to develop bespoke solutions that meet the stringent demands of sectors like IoT, automotive, aerospace, and telecommunications. By incorporating RISC-V cores tailored to their specific needs, companies can achieve significant performance and efficiency gains.

1.2.3 Booming Ecosystem Supports RISC-V ASIC Design

RISC-V's open architecture fosters not only hardware customization but also the development of a comprehensive ecosystem. This ecosystem plays a critical role in facilitating the broad adoption of RISC-V within ASIC design by providing essential tools and resources that streamline the development process. An examination of the following elements reveals the key components of this maturing ecosystem:

- **Comprehensive Development Tools**: A growing suite of development tools caters specifically to RISC-V-based ASICs. This includes tools for Register Transfer Level (RTL) design, verification, synthesis, and physical design, ensuring a smooth workflow for ASIC designers.
- **Diverse IP Core Offerings**: Companies specializing in RISC-V IP cores are providing a rich set of options for ASIC integration. These offerings encompass various configurations of RISC-V cores, peripheral IP (Intellectual Property) blocks, and interconnect IP. This comprehensive library empowers ASIC designers to efficiently build complex systems-on-chip (SoCs) tailored to their specific needs.

1.2.4 Prominent Players in the Maturing RISC-V Ecosystem

The pre-study identified several prominent players shaping the maturing RISC-V ecosystem:

- **SiFive**: A leading provider of RISC-V IP cores, SiFive offers a comprehensive portfolio of customizable processor designs catering to diverse application needs.
- **Andes Technology**: This company specializes in developing high-performance, low-power RISC-V processor cores for a wide range of applications, including those in the Internet of Things (IoT), automotive, and Artificial Intelligence (AI) sectors.
- **NVIDIA**: Following its acquisition of Arm, NVIDIA's growing interest in RISC-V signifies its potential as a viable open-source architecture.
- **Microchip Technology**: Microchip contributes to the RISC-V ecosystem by offering microcontrollers based on the RISC-V architecture, targeting applications in the IoT and embedded systems domains.

1.2.5 Acceleration of Prototyping and Time-to-Market:

RISC-V's open architecture offers significant advantages in terms of development time and prototyping for ASIC design:

- **Streamlined Prototyping**: The open nature of RISC-V allows designers to experiment with various configurations and architectural choices readily. This flexibility fosters faster prototyping cycles, enabling rapid design exploration and optimization. As a result, companies can iterate on their designs quickly, accelerating the product development process.
- **Agile Development Compatibility**: RISC-V aligns well with agile development methodologies. Its open nature facilitates quick modifications and easier integration of feedback from stakeholders during the design phases. This iterative approach reduces the time needed to refine the design and ultimately shortens the time-to-market for ASIC products.

1.2.6 Cost-Effectiveness and Scalability with RISC-V

RISC-V presents compelling advantages for ASIC design in terms of both cost and scalability:

- **Reduced Development Costs**: Unlike proprietary architectures that require licensing fees, RISC-V's open-source nature eliminates these upfront costs, making it a more cost-effective option for developers.

- **Scalable Architecture**: RISC-V offers remarkable scalability. The same core architecture can be adapted to create a wide range of solutions, from resource-constrained, low-power designs ideal for IoT devices to high-performance processors suitable for data centers. This scalability allows ASIC designers to leverage a familiar architecture while tailoring it to meet the specific needs of their application without significant architectural overhauls.

1.2.7 Challenges and Considerations for RISC-V Adoption

While RISC-V presents enticing opportunities, it's crucial to acknowledge the challenges associated with its adoption in ASIC design:

- **Established Market Competitors**: The dominance of entrenched architectures like x86 and Arm in specific markets presents a significant hurdle for RISC-V's widespread adoption. Overcoming these established players and their mature ecosystems requires ongoing innovation and industry support for RISC-V.
- **Intellectual Property Considerations**: Although RISC-V boasts an open-source core, challenges related to intellectual property can still arise when integrating RISC-V cores into commercial products. Careful assessment and management of IP licensing terms associated with specific RISC-V implementations are necessary.
- **Verification and Validation Complexity**: Verifying complex ASIC designs remains a significant challenge, regardless of the chosen ISA. Integrating RISC-V processors necessitates robust verification strategies to ensure the design's correctness and reliability. Designers need to invest in thorough verification methodologies for RISC-V-based ASICs.
- **Evolving Ecosystem**: The RISC-V ecosystem, while rapidly maturing, lacks complete maturity in certain areas. Aspects like optimized libraries, established design flows, and comprehensive tool support are still under development. ASIC designers must carefully evaluate the maturity of the RISC-V ecosystem in relation to their specific needs and project timelines.

1.3 Wish Specification

This project focuses on developing a RISC-V core specifically designed for embedded systems and custom hardware projects. The core prioritizes achieving a balanced trade-off between three critical factors:

• **Performance**: The core should deliver efficient processing capabilities.

• **Power Consumption**: Low power consumption is crucial for battery-powered applications.

• **Area Footprint**: Minimizing the core's physical size on a chip optimizes cost and resource utilization.

The RISC-V core will leverage a Harvard architecture to enable simultaneous instruction and data memory accesses. This approach enhances overall processing efficiency. Additionally, an optimizing folded 6-stage pipeline will be implemented. This pipeline maximizes overlap between execution stages and memory accesses, reducing stalls and improving performance.

Optional features include Branch Prediction, Instruction Cache, Data Cache, Debug Unit and optional Multiplier/Divider Units. Parameterized and configurable features include the instruction and data interfaces, the branch-prediction-unit configuration, and the cache size, associativity, replacement algorithms and multiplier latency. Providing the user with trade offs between performance, power, and area to optimize the core for the application.

1.3.1 Key Specifications:

1.3.1.1 High Performance 32-bit CPU design with Optional Extensions

- **Parameterized 32/64bit data**: The core will be a high-performance 32-bit CPU design with the option for parameterized 32/64-bit data handling.
- **Fast and Precise Interrupts**: The core prioritizes efficient interrupt handling, ensuring timely responses to critical events within the system.
- **Single-Cycle Execution (Optional)**: Single-cycle execution can be included as an option for applications requiring the highest performance. However, this approach may require careful consideration of trade-offs with power consumption and area footprint.
- **Optimizing Folded 6-Stage Pipeline**: This core implements an optimized folded 6-stage pipeline to maximize overlap between instruction execution and memory access stages.

This approach minimizes stalls and improves overall processing efficiency.

- **Memory Protection Support**: The core will offer memory protection support for enhanced system security.
- **Optional/Parameterized Caches**: Instruction and data caches can be optionally integrated to reduce memory access latency for performance-critical applications. The size, associativity, and replacement algorithms of these caches will be configurable.

1.3.1.2 Area and Power Efficiency

The design will prioritizes both minimizing the core's physical footprint (area) and reducing its power consumption:

- **Parameterized Data Width**: The data path width can be configured, allowing users to optimize the balance between performance and power consumption for their specific application. Wider data paths can process more data per cycle but require more hardware resources and consume more power.
- **Clock Gating Logic**: The core will employ clock gating logic to reduce dynamic power consumption. This technique essentially shuts off the clock signal to inactive portions of the core, minimizing unnecessary power usage.
- **Small Silicon Footprint**: The design prioritizes a compact layout to minimize the core's physical size on the chip. This not only reduces manufacturing costs but also allows for more efficient integration with other components on the same die.

1.3.1.3 Multicycle Architecture approach

- **Reduced Area Footprint**: Due to the absence of complex pipeline hazard detection and resolution logic, multicycle architectures require less silicon area. This can be crucial for resource-constrained embedded systems or designs targeting low-cost fabrication processes.
- **Synchronous Memory Compatibility**: Multicycle architectures are well-suited for use with synchronous memory, which is the prevalent type of memory found in most real-world applications, including FPGAs with block RAM. Synchronous memory offers reliable data transfers by coordinating operations with a clock signal.

- **Pipelined Architecture**: Pipelined designs increase performance by having several instruction "in fly" at the same time. But this also means there is some kind of "out-of-order" behavior: if an instruction at the end of the pipeline causes an exception all the instructions in earlier stages have to be invalidated.

1.3.1.4 Instruction Set Architecture (ISA)

The RISC-V core will support the base integer instruction set architecture (ISA) defined by the RISC-V specification. This core ISA provides a foundation for efficient execution of various workloads.

- **Basic Integer Instructions**: The core will support fundamental instructions for arithmetic, logical, data transfer, and control flow operations. This includes essential instructions like branch, jump, and call instructions for program branching and subroutine management.
- **Optional Branch Prediction**: A branch prediction algorithm can be optionally integrated to improve the efficiency of branch instructions. Branch prediction attempts to forecast the outcome of a conditional branch instruction, potentially reducing pipeline stalls and improving overall performance.
- **Custom ISA Extension Support (Optional)**: The design may offer the flexibility to incorporate user-defined RISC-V instruction extensions. These extensions can be tailored to specific application requirements, potentially accelerating performance for specialized tasks. However, the decision to include custom extensions requires careful consideration of development complexity and potential compatibility limitations.

Bibliography

[1] RISC-V International: https://riscv.org/

[2] BBC Research: https://www.bccresearch.com/market-research/semiconductor-manufacturing/global-risc-v-technology-market.html

[3] https://semico.com/content/risc-v-cpu-market-analysis-sip-socs-ai-and-design-starts

[4] https://riscv.org/technical/specifications/

[5] https://wiki.riscv.org/display/HOME/RISC-V+Technical+Specifications